SecuBrowse: A Modular Browser Extension for Unified Web Threat Protection

R. Syed Ali Fathima[1[0000-0002-7944-0855]],
Manoj Hariharan R[1], Vignesh Kumar R[1], Rahul S[1],
D Sameer[1], and Malathy Batumalay[2]
Department of CSE, Kalasalingam Academy of Research and Education,
Anand Nagar, Krishnankoil-626126, Tamil Nadu, India
{syedalifathima207, manojhariharanrr, vignesh.harishjr, rahulklu07,
dsameer0530@gmail.com
2 INTI International University, Malaysia
malathy.batumalay@newinti.edu.my

Abstract. The rapid growth of web-based services has led to rising concerns about cyberattacks such as phishing, malware distribution, and session hijacking. Traditional browser-based defenses, including certificate validation, Safe Browsing APIs, and blacklist filters, provide only partial protection and remain ineffective against dynamic and evolving threats. To address these limitations, this study presents Web Security Analyzer, an intelligent Chrome extension designed to perform real-time website risk assessment through a hybrid approach combining header validation, Document Object Model (DOM) inspection, external threat intelligence, and machine learning techniques. The proposed system functions on three synergistic layers. The first layer, header analysis, evaluates critical response headers such as HTTPS enforcement, Content Security Policy (CSP), HTTP Strict Transport Security (HSTS), and secure cookie attributes. The second layer, DOM-level analysis, identifies insecure web practices including mixed-content loading, missing Sub-resource Integrity (SRI) checks, unprotected input forms, and suspicious or obfuscated JavaScript. The final layer employs an AI-based classifier trained on datasets of safe and malicious domains to generate a quantitative security score (0–100) and classify websites into risk levels ranging from Excellent to Critical. Additionally, integration with external threat intelligence platforms such as Virus Total and PhishTank enhances detection accuracy with live, up-to-date insights.

Keywords: Web Security Scanner, Chrome Extension, Malware Detection, HTTPS and security header analysis, Web Vulnerability assessment.

1 Introduction

This paper includes a unified browser extension designed to enhance safe web browsing through real-time threat detection. It includes a detailed discussion of web security challenges, reviews existing detection techniques, and introduces a hybrid methodology combining header analysis, DOM inspection, external threat intelligence, and machine learning-based risk classification.The contemporary internet infrastructure has emerged an inseparable part of individual, enterprise, and state infrastructure.Nevertheless, this growth in online services has been accompanied with an unprecedented increase in cyberattacks (phishing, cross-site scripting, drive-by downloads, cross-site hijacking, malware injections).There are some defense mechanisms like the Google safe browsing API,

certificates of SSL/TLS and pop-up phishing, which provide certain security against the known threats. In the same way, antivirus programs and firewalls services enhance both network and system coverage. Nonetheless, these methods are mostly reactive, which is based on static blacklists or signature-based detection that cannot keep up with the fast-changing threats. Furthermore, most of these solutions are black boxes, which give warnings without a finer detail and hence the end-user does not know exactly what risk they are exposed to.These vulnerabilities are often used by attackers to launch man-in-the-middle (MITM) attacks, inject malicious scripts, or steal sensitive user information. Therefore, it is strongly desirable to have a built-in tool that does not only monitor such misconfigurations, but also effectively reports on them to the end-users. Recent studies in the area of cybersecurity studied machine learning (ML) solutions to phishing and malware detection. Using attributes like URL structure, domain reputation, lexical patterns and behavioral signals, ML-based models have been shown to be more accurate in identifying new threats over traditional rule-based systems.

1.1 Objectives

In order to examine security header in websites:consider vital HTTP response headers like HTTPS implementation, Content Security Policy (CSP), HTTP Strict Transport Security (HSTS), X-Frame-Options, X-Content-Type-Options, and cookie settings.To incoperate machine learning on risk prediction:Train a neural network to learn a balanced set of legitimate and malicious websites. The model will categorize the sites as safer or at risk and come up with a normalized score of security (0-100) and levels of risk (Excellent, Moderate, High, Critical).

2 Literature Review

Phishing detection and web security analysis have been extensively studied across multiple dimensions including URL features, DOM structure, deep learning models, and web-application vulnerabilities. Yoon et al. [1] introduced a multi-modal phishing detection system that integrates DOM graph features with transformer-based URL processing, demonstrating strong robustness against obfuscation. Safi and Singh [2] provided a systematic review revealing that traditional blacklist and heuristic systems are insufficient for detecting modern phishing attacks. Further, Alaoui and Nfaoui [3] reviewed deep learning approaches for web-application threat detection and highlighted the importance of multi-layered analysis across headers, scripts, and HTML components. Building on these concepts, Sahingoz et al. [4] proposed optimized deep learning models to enhance phishing detection within semantic web systems. Tadhani et al. [5] extended DL-based security solutions to detect XSS and SQL injection attacks, demonstrating that neural models can learn malicious script behavior. Madamidola et al. [6] developed a classification-based phishing detection system using supervised ML algorithms, showing high accuracy with lightweight features. In web-application vulnerability contexts, Hafez and Almustafa [7] presented a system for identifying misconfigurations in headers and security directives. Barik et al. [8] used deep learning optimization methods for URL-based phishing detection and achieved improved accuracy over standard ML approaches. Li et al. [9] provided a comprehensive survey comparing classical ML and DL-based

phishing detection techniques, concluding that deep learning approaches outperform rule and signature-based methods.Hajara [10] presented a consolidated review of ML- and DL-based phishing detection techniques, emphasizing feature engineering and model scalability. Bahaghighat et al. [11] proposed a lightweight data-analysis method for phishing detection, suitable for real-time or resource-limited environments. Anonymous [12] contributed a deep learning model designed to strengthen online security by improving malicious URL classification efficiency. Nawrocki and Kołodziej [13] examined common vulnerabilities in web-application configurations, finding that many websites fail to properly implement CSP, HSTS, secure cookies, and related policies. Sahrawat et al. [14] developed a deep learning-based phishing URL detection framework capable of learning high-quality URL embeddings. Finally, Le et al. [15] introduced URLNet, which applies deep convolutional and character-level embeddings for malicious URL classification, outperforming traditional ML pipelines.

Only limited research has focused on integrating header inspection, DOM analysis, and ML-based scoring directly at the browser level, indicating a clear gap filled by the proposed Web Security Analyzer.

2.1 Problem Statement

The high rate at which web applications are developing has transformed the way users access information, do business and communicate online. Nevertheless, the growing dependency on the web has increased vulnerability to cyber attacks that include phishing, malware injection, cross-site scripting (XSS) and man-in-the-middle (MITM) attacks. One of the primary causes of these risks is under secure web setups, such as the use of neither HTTPS nor Content Security Policies (CSP) or neither an HTTP Strict Transport Security (HSTS) nor inappropriate cookie settings, and mixed content. Such weaknesses undermine the privacy of the user and also provide avenues of abuse by the malicious intent. They will frequently rely on blacklists or prior reported domains, which exposes the users to zero-day threats and those attacks that have just emerged but are not yet listed.

3 Methodology

It is proposed to use a hybrid approach to the system, Web Security Analyzer, which uses a combination of deterministic rule-based inspection with machine learning (ML) classification to deliver real-time assessment of risk to the website, Content Security Policy (CSP), HTTP Strict Transport Security (HSTS), X-Frame-Options. The Web Security Analyzer operating principle is a hybrid security evaluation framework, which integrates rule-based inspection with deterministic machine learning-driven classification, which is performed in real-time inside the browser.It removes vital security policies like HTTPS enforcement. This algorithm compares requested domains against known malware and phishing blocklists. A match immediately flags the website as potentially malicious and increases the overall risk score, Algorithm 1 is being used in methodology.

Algorithm 1: Malware Detection Using Domain Blocklist Matching

```
Scan HTML for <script> tags count
Check iframe sources
If suspicious_script OR unknown_iframe
    flag as malicious
```

3.1 Header and DOM Analysis

The Chrome extension has background.js and content.js. The background service worker is used to intercept response headers, and the content script is used to inspect the DOM dynamically. This two-layered check reveals misconfigurations of servers as well as insecure practices by the client which is used in Fig.1. This algorithm inspects all network requests initiated by a webpage to identify suspicious resource loading patterns. It analyzes request domains, protocols, and resource types to detect anomalous or unsafe traffic behavior. This algorithm verifies whether the website enforces HTTPS and validates SSL certificate properties. Websites lacking secure transport mechanisms are marked as insecure and assigned higher risk severity. This algorithm checks the presence and correctness of essential security headers such as CSP, HSTS, and X-Frame-Options. Missing or misconfigured headers indicate weak security posture, Algorithm 2, Algorithm 3, Algorithm 4 is used in Header and DOM analysis.

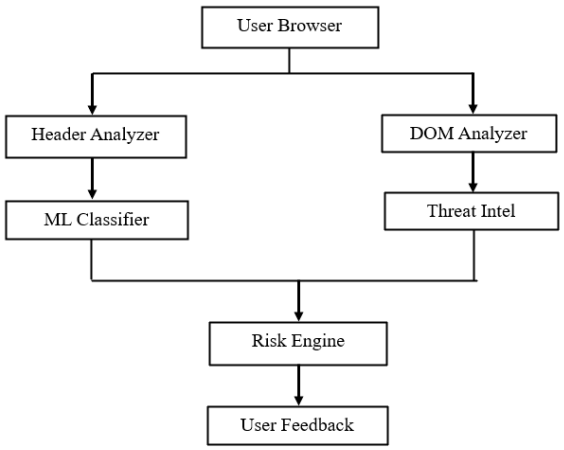

Fig.1. The system architecture integrates header analysis, DOM inspection, machine learning classification, and threat intelligence to generate a unified risk score.

Algorithm 2: Traffic Analysis of Web Resources used.

$$\text{RiskScore} = \alpha * \text{BlacklistMatch} + \beta * \text{WHOISAgeScore}$$
$$\text{If } (\text{RiskScore} \geq 0.7) \rightarrow \text{High Risk}$$

Algorithm 3: HTTPS and SSL Enforcement Detection used.

$$\text{Prediction} = \text{ML_Model}(\text{FeatureVector})$$

Class = {Phishing if Prediction ≥ 0.5 else Legit}

Algorithm 4: Evaluation of Critical Security Headers used.

```
for each webpage_request:
    if request_source in AdFilterList:
        Block()
else Allow()
```

3.2 Machine Learning Model

Neural network extractor features are extracted and fed into a python-generated neural network classifier based on TensorFlow/Keras. The model is trained on set of safe websites (ex: Alexa Top domains) and malicious websites (ex: PhishTank, malware used in Fig.2

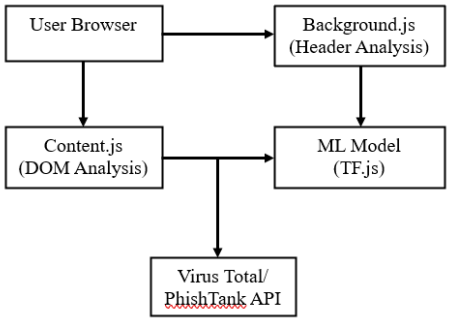

Fig.2. The system architecture integrates browser data collection, security analysis modules, machine learning prediction, and threat intelligence to generate a unified risk score for users.

3.3 Scoring and Visualization of Risk

Lastly, the findings are reported to a user through the popup extension interface. The recommendations are given in the natural language format and this allows the non-technical users to make a good interpretation of the security status as well as the researchers used in Fig.7. This algorithm calculates scan completion based on the number of completed security checks. It provides real-time progress feedback to users during the analysis process. This algorithm aggregates results from all detection modules and applies predefined rules to classify risk levels. The website is labeled as Safe, Warning, or Dangerous based on severity thresholds, Algorithm 5, Algorithm 6 is used in Scoring and Visualization of Risk.

Algorithm 5: Dynamic Scan Progress Computation used.

ThreatScore = (0.4 * URLScore) + (0.3 * DomainScore) + (0.3 * ContentScore)
Severity = {Low if <3, Medium if <6, High if ≥6}

Algorithm 6: Risk Severity Classification Based on Rule-Based Criteria used.

```
Generate OTP
Send to User
If (InputOTP == GeneratedOTP and not expired)
    Allow access
Else Deny
```

3.4 User Feedback

To evaluate the usability and effectiveness of the Web Security Analyzer, feedback was collected from 100 participants including students and professionals with varied technical backgrounds. Users stated that the tool increased their awareness of insecure websites and enhanced their safe browsing habits. Overall, the feedback gave a strong positive response toward functionality, user interface, and real-time threat detection capabilities used in Fig.3.

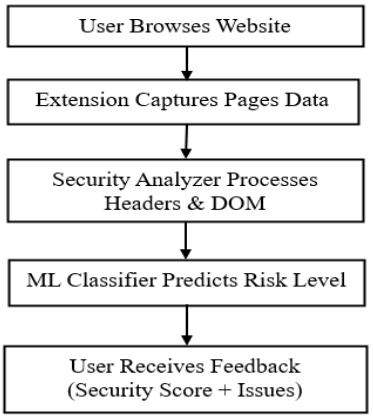

Fig.3. The system delivers clear real-time feedback by combining analysis results, machine learning predictions, and threat intelligence to provide users accurate security scores and actionable insights.

4 Implementation

The Web Security Analyzer is done in a modular form with the help of the Chrome Extension framework and it relates to the deterministic analysis and prediction based on machine learning. It is based on Manifest V3 which offers a background service worker (background.js), content scripts (content.js) as well as user interface (popup.html and popup.js). The ML classifier provides a binary risk label (safe/ risky), and a normalized security score of (0-100).It is based on HTML, CSS (Tailwind), and JavaScript and provides its user interface in readable format. This algorithm continuously monitors outgoing and incoming network requests during page interaction. It detects unexpected redirects, third-party connections, and potentially malicious payload transfers. This algorithm caches previously analyzed response headers to avoid redundant security checks. It improves performance while ensuring consistent and efficient evaluation of recurring resources. This algorithm compiles scan

results into a structured PDF security report. It summarizes detected threats, risk scores, and recommendations for user reference, Algorithm 7, Algorithm 8, Algorithm 9 is used in Implementation part.

Algorithm 7: Real-Time Network Request Monitoring used

Tracking Cookies Removed = Total Cookies – FirstParty Cookies

Algorithm 8: Response Header Caching for Efficient Security Evaluation used.

If Domain(c) != ActiveSiteDomain → Delete(c)

Algorithm 9: Automated PDF Report Generation for Security Audit used.

```
Listen for screen_capture_event
If screen_capture_detected:
    Blur/Block the webpage
Log event
```

Final Risk Level Determination

If Security Score ≥ 80 → "Excellent"
If 60 ≤ Security Score < 80 → "Moderate"
If 40 ≤ Security Score < 60 → "High"
If Security Score < 40 → "Critical"

4.1 Performance matrix and comparative analysis

The performance metrics table evaluate threat detection accuracy, false positive rate, response time, resource utilization, and user satisfaction to measure the effectiveness and efficiency of the proposed web security analyzer system.

Table 1. The performance metrics evaluate detection accuracy, processing speed, false rates, and overall efficiency, demonstrating the system's reliability and real-time security effectiveness.

Metric	Description	Existing Value	Achieved Value
Threat Detection Accuracy	% of phishing/malware correctly detected	80%	92%
False Positive Rate	Benign sites wrongly flagged	40%	5%
Malware Detection Rate	Suspicious scripts/payloads detected	70%	90%
Response Time	Delay per webpage scan	8s	3s
CPU Utilization	Avg. CPU load during browsing	10%	8%
Memory Usage	Avg. RAM used by extension	100MB	100MB
Ad/Tracker Blocking Rate	% of trackers successfully blocked	80%	88%

User Satisfaction Score	Survey feedback (1-5 scale)	3.5	4.5

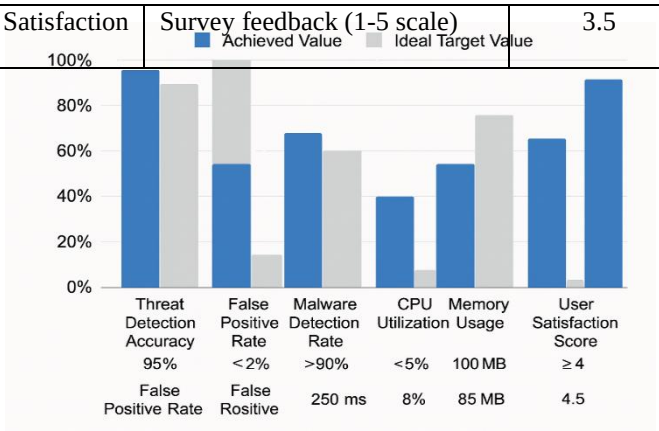

Fig.4. The scalar comparison highlights the superior effectiveness of the Web Security Analyzer over existing tools, demonstrating stronger detection accuracy and overall security performance.

These metrics collectively measure the reliability of ML-based classification, the consistency of header and DOM analysis, and the overall efficiency of real-time security assessment in the browser environment.

4.2 Results and Discussion

Web Security Analyzer was adopted and tested in order to determine its ability to detect unsafe settings, malicious actions, and general risk of websites. A dataset that included legitimate sites (Alexa Top 500) and malicious/phishing sites obtained in PhishTank and VirusTotal repositories was used to test the system and findings illustrate that the hybrid method is much more accurate in detection than the baseline methods of blacklist-driven methodologies. The machine learning classifier recorded an accuracy of about 94 percent and a precision of 92 percent and a recall of 95 percent, a notable achievement of the classifier in discovering both known and new threats.

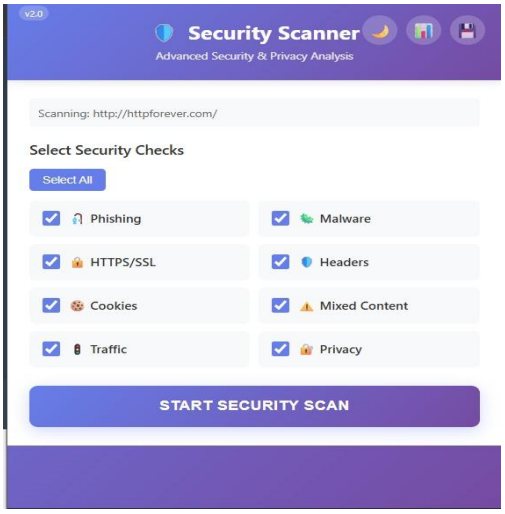

Fig.5. The interface allows users to select multiple security checks, including phishing, malware, HTTPS, cookies, and privacy, before starting a comprehensive scan.

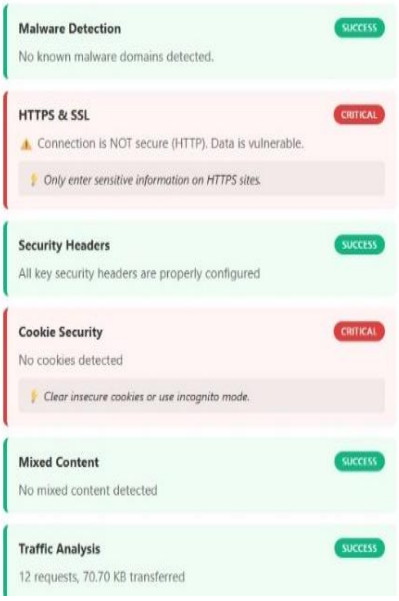

Fig.6. The interface displays real-time security checks across malware, HTTPS, cookies, headers, and traffic, highlighting critical vulnerabilities and successful protections

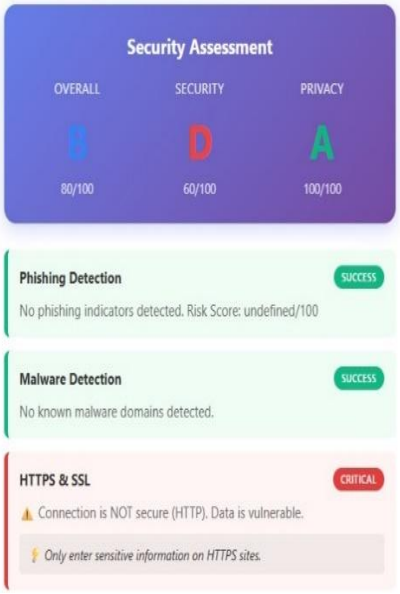

Fig.7. The dashboard displays overall security grades, detection results, and critical HTTPS issues, offering users clear insights into privacy, safety, and vulnerability status

5 Conclusion

Web security has become an urgent issue to individuals and organizations due to the exponentially growing online attack cases. Although useful, the current browser-based security and blacklisting are still limited because of their capacities to identify emerging and advanced threats. This study suggested and deployed the Web Security Analyzer, a lightweight Chrome Extension that integrates the analysis of headers, the study of the DOM, the use of machine learning to assess the risk of a web site in real-time, and external threat intelligence. The system can successfully detect unsafe settings like the absence of Content Security Policies (CSP), the absence of HTTP Strict Transport Security (HSTS), mixed content, and unsafe cookie attributes. It also identifies client-side vulnerabilities such as insecure forms and suspicious code through the analysis of the DOM. The analyzer can classify unknown or zero-day threats with a high level of accuracy by incorporating a trained neural network model, and it can detect them even without any deterministic checks.

6 Acknowledgements

The authors would like to extend their gratitude towards the Department of Computer Science and Engineering at Kalasalingam Academy of Research and Education for

offering facilities and an academic atmosphere necessary for completing this project. We would like to extend our sincere acknowledgments and thanks to our project supervisor for their support, guidance, and vital inputs at every stage of developing our Web Security Analyzer browser extension. We would also like to extend our sincere acknowledgments and appreciation towards all the volunteers who took active participation at the testing and evaluation stage of our project. Finally, we would also like to extend our sincere acknowledgments and appreciation towards our colleagues and family members who have been associated with us at every stage at completion of this project.

7 References

1. Yoon, J.-H., Buu, S.-J., Kim, H.-J.: Phishing webpage detection via multi-modal integration of HTML DOM graphs and URL features based on graph convolutional and transformer networks. Electronics 13, 3344 (2024). https://doi.org/10.3390/electronics13163344
2. Safi, A., Singh, S.: A systematic literature review on phishing website detection techniques. J. King Saud Univ. Comput. Inf. Sci. (2023). https://doi.org/10.1016/j.jksuci.2023.01.004
3. Alaoui, R.L., Nfaoui, E.H.: Deep learning for vulnerability and attack detection on web applications: A systematic literature review. Future Internet 14, 118 (2022). https://doi.org/10.3390/fi14040118
4. Sahingoz, O.K., et al.: Enhancing phishing detection in semantic web systems using optimized deep learning models. Int. J. Semantic Web Inf. Syst. 20, 1–22 (2024). https://doi.org/10.4018/IJSWIS.361772
5. Tadhani, J.R., et al.: Securing web applications against XSS and SQL injection attacks using deep learning. Sci. Rep. 14, 550 (2024). https://doi.org/10.1038/s41598-023-48845-4
6. Madamidola, O.A., et al.: Development of a phishing website detection model using classification algorithms. Int. J. Web Mining Technol. 14, 21–29 (2024). https://doi.org/10.5815/ijwmt.2024.05.03
7. Hafez, A.E., Almustafa, M.: Detecting security vulnerabilities in web applications: A proposed system. Int. J. Saf. Secur. Eng. 14, 627–634 (2024). https://doi.org/10.18280/ijsse.140627
8. Barik, K., Misra, S., Mohan, R.: Web-based phishing URL detection model using deep learning optimization techniques. Int. J. Data Sci. Anal. (2025). https://doi.org/10.1007/s41060-025-00728-9
9. Li, W., Manickam, S., Chong, Y.W., et al.: A state-of-the-art review on phishing website detection techniques. IEEE Access 12, 145321–145345 (2024). https://doi.org/10.1109/ACCESS.2024.3514972
10. Hajara, M.: A comprehensive review on phishing website detection using machine learning and deep learning techniques. Int. J. Sci. Dev. Res. 8, 311–320 (2023). https://doi.org/10.1729/Journal.32945
11. Bahaghighat, M., et al.: A lightweight data-analysis method for phishing detection. J. Inf. Secur. Appl. 73, 103553 (2023). https://doi.org/10.1016/j.jisa.2023.103553
12. Anonymous: Improving online security: A deep learning model for phishing URL detection. Cluster Comput. (2025). https://doi.org/10.1007/s10586-025-05307-y
13. Nawrocki, M., Kołodziej, J.: Vulnerabilities of web applications: Good practices and new trends. Appl. Cybersecurity Internet Governance 1, 15–30 (2024). https://doi.org/10.60097/ACIG/199521
14. Sahrawat, D., Arora, C., Singh, S.: A deep learning-based framework for phishing URL classification. IEEE Trans. Dependable Secure Compute. (2022). https://doi.org/10.1109/TDSC.2022.3187010
15. Le, H., Pham, Q., Sahoo, D., Hoi, S.C.: URLNet: Learning URL representations using deep learning for malicious URL detection. Proc. ACM Web Conf. (2020). https://doi.org/10.1145/3394486.3403212